Lean through Artform Calculus

An introduction to Lean, and to the book's theorems in Lean

Part I explains the basics of Lean, a programming language in which proofs are checked by the computer. Part II writes the constructions of Chapter 1 of *Artform Calculus* in Lean. Part III introduces the repository that holds this guide, where the book's numbered statements are written in Lean with Mathlib and proved, and reads four of them closely. References such as Section 1.4, Figure 1.9 or Definition 1.5 point into the book; their numbers are those of its 52nd edition (29 September 2026). Every listing is cut from files that Lean 4.34.1 accepts as a whole: two files in core Lean for Parts I and II, and for Part III a file that imports the repository's library, and the library itself.

Contents

Part I. The basics of Lean **4**

Running Lean 4

Values and types 5

Definitions and functions 6

Definitions by cases, and recursion 7
Inductive types 8
Structures 9
Types that depend on values 9
Propositions are types 10
Tactics 11
Arguments that Lean finds 13
Reading Lean's messages 13

Part II. Chapter 1 of the book in Lean **15**
From the book's notation to Lean 15
Rods: their rules and their laws 16
Witnesses, and three equalities in $2 + 3 = 5$ 18
Laws as scripts 19
A tagged choice and a complete assignment 20
Formation lines 21
Time 23
Meaning 26
Spaces and states 28
Scripts and roles 29
What Lean does not see 31
Every formation line in Lean 31
The same constructions in the library 32

Part III. The textbook's theorems in Lean **35**
What the repository proves 35
Getting and building it 37
From core Lean to Mathlib 37
How the book is written in Lean 40

Four statements read closely 41
How the repository checks itself 55
Following a proof, and reporting a problem 57

Appendix. The book's statements in the repository **58**

Bibliography **65**

Part I. The basics of Lean

Lean is a programming language in which programs and proofs are written in one notation and checked by one program. A Lean file is a list of declarations. Lean reads the file from top to bottom, and for each declaration it either accepts it or says what is wrong with it. This part teaches enough of the language to read Part II, which writes the constructions of Chapter 1 of the book in Lean. It assumes no experience of programming.

Every example was checked by Lean 4.34.1. A comment after `--` at the end of a line shows what Lean answers to that line. Where Lean refuses a line, its message follows the listing, as Lean prints it.

Running Lean

The quickest way to start needs nothing installed. The Lean playground at `https://live.lean-lang.org` is an editor in the browser: type on the left, and Lean answers on the right. For longer work, install the editor VS Code and its official Lean 4 extension. The extension installs Lean itself and checks a file while it is being typed. Lean's answers appear in a panel beside the text, called the *infoview*, and an error is underlined in red where it occurs. Lean also runs in a terminal: `lean Basics.lean` checks the file `Basics.lean` and prints the answers and the errors.

The examples of this guide are in the folder `textbook/tutorial` of the repository that holds it. Part I's are in `LeanBasics.lean` and Part II's in `ArtformChapter1.lean`. Those files check every answer quoted here, so an editor shows nothing while the answers are right. The folder `play` holds copies without the checks, which show every answer in the infoview. To try them, copy `play` to a folder outside the repository and open it in VS Code; inside the repository, the editor would first set up the whole project, Mathlib included.

The examples use Lean's core language only. Most mathematicians who use Lean also use Mathlib, a large library of mathematics written in Lean; nothing here needs it.

Values and types

Every expression in Lean has a type. Two commands ask about an expression: `#check` shows its type, and `#eval` computes its value.

```
#eval 2 + 3                 -- 5
#check 2 + 3                -- 2 + 3 : Nat
#eval "glass " ++ "bead"    -- "glass bead"
#check (2, "two")           -- (2, "two") : Nat × String
#check [1, 2, 3]            -- [1, 2, 3] : List Nat
#eval [1, 2, 3].length      -- 3
```

The colon is read *has type*, as in the book: `2 + 3 : Nat` says that $2 + 3$ is a natural number. `Nat` is the type of the natural numbers, `String` of texts, `Nat × String` of pairs of a number and a text, and `List Nat` of lists of numbers. A text is written between double quotes, and ++ joins two texts.

Numbers need care. `Nat` has no negative numbers, so subtraction stops at zero, and division of natural numbers drops the remainder, which `%` gives. To compute in another type, write the expression

with its type, `(e : T)`. On the integers `Int` subtraction goes below zero, and on the rationals `Rat` division is exact:

```
#eval 7 - 10              -- 0
#eval (7 - 10 : Int)      -- -3
#eval 7 / 2               -- 3
#eval 7 % 2               -- 1
#eval (1/4 + 1/3 : Rat)   -- (7 : Rat)/12
```

The last line is the sum $\frac{1}{4} + \frac{1}{3} = \frac{7}{12}$ of Figure 1.16. Lean prints a rational with its type.

Definitions and functions

`def` gives a name to a value. The name comes first; then the inputs, each in parentheses with its type; then a colon and the type of the result; and then `:=` and the value.

```
def double (n : Nat) : Nat := 2 * n
#eval double 21           -- 42
#eval double (double 3)   -- 12
#check double             -- double (n : Nat) : Nat
```

A function is applied by writing it before its input, without parentheses: `double 21`. Parentheses only group, as in `double (double 3)`. `#check` of a defined name shows its inputs and the type of its result.

A function is a value, and `fun` builds one without a name: `fun n => 2 * n` is the function $n \mapsto 2n$, of type `Nat → Nat`. A function of two inputs takes them one at a time, so `add 2` is again a function: the one that adds two to its input.

```
#check fun n : Nat => 2 * n  -- fun n => 2 * n : Nat→Nat
def add (m n : Nat) : Nat := m + n
#check add 2                 -- add 2 : Nat→Nat
def twice {α : Type} (f : α→α) (x : α) : α := f (f x)
#eval twice double 3         -- 12
```

The first input of `twice` is a type, α. The braces make it *implicit*: Lean works out α from the other inputs, here `Nat`, and the caller does not write it. `twice double 3` doubles 3 twice.

Definitions by cases, and recursion

A definition may be given case by case, with one line for each form the input can take. A natural number is either 0 or $n+1$ for a smaller number n, and on the line for $n+1$ the definition may use itself at n:

```
def sumTo : Nat→Nat
  | 0     => 0
  | n + 1 => (n + 1) + sumTo n
#eval sumTo 4  -- 10
```

So `sumTo 4` is $4+3+2+1+0 = 10$, the ten points of the tetractys (Section 1.6). Lean accepts such a definition only after two checks: every case is covered, and every call of `sumTo` inside it is at a smaller number, so that the computation ends. A definition that leaves a case out is refused:

```
def isZero : Nat→Bool
  | 0 => true
```

Lean refuses it:

```
Missing cases:
(Nat.succ _)
```

The message names the case that was left out: a number of the form $n+1$, which Lean writes as `Nat.succ _`.

Inductive types

A new type is made by listing the ways to build its values. Each line names a *constructor* and says what it takes. A colour is light or dark, and a tree is either a leaf carrying a letter or a node joining two trees:

```
inductive Colour where
  | light
  | dark
inductive Tree where
  | leaf : Char → Tree
  | node : Tree → Tree → Tree
```

Every value of the type is built by its constructors, and only by them; Section 1.11 calls them the introduction rules. A function on a type is defined by cases, one for each constructor, and on a node it may use itself on the two smaller trees. The tree built from a, b and c below has three leaves, as the construction trees of Section 1.4 have one leaf for each atom.

```
def Tree.size : Tree → Nat
  | .leaf _   => 1
  | .node s t => s.size + t.size
def abc : Tree := .node (.leaf 'a') (.node (.leaf 'b') (.leaf 'c'))
#eval abc.size  -- 3
```

Two pieces of notation appear here. `Tree.size` is the function `size` belonging to the type `Tree`, and for a tree `t`, `t.size` means `Tree.size t`. Where Lean already knows that a tree is meant, `.leaf` means `Tree.leaf`. The natural numbers are an inductive type too, with the two constructors `zero` and `succ`, and the cases `0` and `n + 1` of `sumTo` are these two.

Structures

A *structure* is a type of records with named fields. A value gives its fields either by name or in order between angle brackets, and a field is read with a dot:

```
structure Interval where
  lo : Nat
  hi : Nat
def i : Interval := { lo := 2, hi := 5 }
def j : Interval := ⟨2, 5⟩
#eval i.hi - i.lo  -- 3
```

A structure is an inductive type with one constructor, which takes all the fields, and `⟨2, 5⟩` applies that constructor.

Types that depend on values

Types are values too: `Nat` has type `Type`. So a function may return a type, and a type may depend on a number. `Fin n` is the type of the numbers below n, the cells of the book's rod R_n. A value of it is a number together with a proof that the number is below n, written between angle brackets. The words `by decide` ask Lean to find the proof itself, by computing.

```
#eval (⟨2, by decide⟩ : Fin 5)  -- 2
#eval (7 : Fin 5)               -- 2
```

The second line shows a trap. A bare numeral in `Fin 5` is read modulo 5, so `7` is accepted and means 2, and nothing checks that 7 is not a cell of R_5. Part II writes cells with their proofs for this reason.

A function whose output type depends on its input is a *dependent function*. `last n` is the largest number below $n + 1$, and its type

`Fin (n + 1)` changes with n. Its proof, `by omega`, asks Lean to show $n < n + 1$ by arithmetic.

```
def last (n : Nat) : Fin (n + 1) := ⟨n, by omega⟩
#check last  -- last (n : Nat) : Fin (n + 1)
def p : (n : Nat) × Fin n := ⟨3, ⟨1, by decide⟩⟩
```

The type of `last` is written `(n : Nat) → Fin (n + 1)`: it is the book's $\Pi_{n:\mathbb{N}}\mathsf{Fin}(n+1)$. In a *dependent pair* the type of the second entry depends on the first: a value of `(n : Nat) × Fin n` is a number n together with a number below it, the book's $\Sigma_{n:\mathbb{N}}\mathsf{Fin}(n)$. The pair `p` is 3 with 1.

Propositions are types

A proposition is a type as well, of the sort `Prop`, and a proof of it is a value of that type. The equation `2 + 3 = 5` is a proposition, and `rfl` is a proof of it, because both sides compute to 5. `example` states a proposition and gives its proof; `theorem` does the same and gives the proof a name.

```
#check (2 + 3 = 5 : Prop)  -- 2 + 3 = 5 : Prop
example : 2 + 3 = 5 := rfl
```

A false proposition has no proof. Asked to decide one by computing, Lean answers that it is false:

```
example : 2 + 2 = 5 := by decide
```

Lean refuses it:

```
Tactic ‘decide‘ proved that the proposition
  2 + 2 = 5
is false
```

The logical connectives are types built from types. A proof of an implication $p \to q$ is a function that turns any proof of p into a proof of q. A proof of $p \wedge q$ is a pair of proofs. A proof of $\exists x,\ P(x)$ is a pair: a witness x together with a proof of $P(x)$. A proof of $\forall x,\ P(x)$ is a dependent function, giving a proof of $P(x)$ for every x. `False` is the proposition with no proof, and $\neg p$ means $p \to$ `False`.

```
example (p : Prop) : p→p := fun h => h
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := ⟨hp, hq⟩
example : ∃ n : Nat, n * n = 9 := ⟨3, rfl⟩
```

The first line proves $p \to p$ by the function that returns its input. The second pairs two proofs. The third gives the witness 3 with the proof that $3 \cdot 3 = 9$. A proof of an existence statement is a witness with its evidence, the shape of every witness in the book.

Tactics

A long proof is easier to build in steps. After `by` come *tactics*: commands that each change the *goal*, the statement still to be proved, until nothing is left to prove. In the editor the infoview shows the goal after each step. These are the tactics this text uses:

Tactic	What it does
`rfl`	proves an equation whose two sides compute to one value
`decide`	checks a statement about finitely many cases by computing each
`omega`	proves facts of integer arithmetic: $+$, $-$, $\leq$, $<$, and products, quotients and remainders by fixed numbers
`intro h`	begins a proof of $p \to q$ by assuming p and naming its proof `h`
`exact e`	finishes the goal with the proof `e`
`cases`, `induction`	split the goal by the constructors of a value
`simp`	rewrites the goal with definitions and known equations
`grind`	searches for a proof by combining the facts at hand

```
example (p q : Prop) : p → (p → q) → q := by
  intro hp hpq
  exact hpq hp
example (a b : Nat) (h : a < b) : a + 1 ≤ b := by omega
example : ∀ n : Fin 10, n.val * n.val % 4 ≠ 2 := by decide
```

The first proof assumes p and $p \to q$, then applies the second to the first. The second is arithmetic. The third checks, for the ten numbers below 10, that a square leaves remainder 0 or 1 on division by 4, never 2.

Induction proves a statement for every natural number from two cases, 0 and $n + 1$; in the second case the statement for n may be used. Here it proves Gauss's formula $2(1 + 2 + \cdots + n) = n(n + 1)$:

```
theorem sumTo_formula (n : Nat) : 2 * sumTo n = n * (n + 1) := by
  induction n with
  | zero      => rfl
  | succ n ih => simp only [sumTo]; grind
```

In the case `succ n` the name `ih` stands for the formula at n. `simp only [sumTo]` unfolds `sumTo` once, at $n+1$, and `grind` finishes the algebra.

Arguments that Lean finds

An input in square brackets is supplied by Lean, not by the caller. `[Add α]` asks for an addition on the type α. Lean has one for `Nat`, `Int` and `Rat`, so one definition serves all three:

```
def sum3 {α : Type} [Add α] (a b c : α) : α := a + b + c
#eval sum3 1 2 3                    -- 6
#eval sum3 (1/2 : Rat) (1/3) (1/6)  -- 1
```

Such an input is called an *instance*, and the family of instances it belongs to a *class*. `decide` works in the same way: it asks for an instance that says how to decide the statement, and Lean comes with instances for numbers, for the types `Fin n`, and for statements built from them. Part II adds one class of its own, `Listed`, for the finite types of Chapter 1.

Reading Lean's messages

The commonest message is a type mismatch. It shows an expression, the type it has, and the type Lean expected there:

```
example : double 2 = "four" := rfl
```

Lean refuses it:

```
Type mismatch
  "four"
has type
  String
but is expected to have type
  Nat
```

Read it from the bottom. Lean expected a `Nat` on the right of `=`, because the left side, `double 2`, is a `Nat`, and it found a `String`. A misspelt name is reported as unknown:

```
#eval doubel 21
```

Lean refuses it:

```
Unknown identifier ‘doubel‘
```

When a message is hard to read, `#check` on the parts of the expression usually shows which type is not the one meant.

Further reading

Avigad, de Moura, Kong and Ullrich teach Lean from its types and tactics, in the order of this part and much further [1]. Christiansen teaches Lean as a programming language [3]. Avigad and Massot teach mathematics in Lean with Mathlib [2].

Part II. Chapter 1 of the book in Lean

This part writes the constructions of Chapter 1 of the book in Lean, in the order of the chapter. Lean's type theory contains the types and rules of Section 1.11 [6], and Lean checks every judgement a program makes. A formation line becomes a declaration, the inhabitant a plate works through becomes a value, and a value of the wrong type is refused with a message.

The listings are cut from one file, which Lean 4.34 accepts as a whole; it uses Lean's core language and no library. A routine part of a value or a proof is printed as $\cdots$, and the file contains it.

From the book's notation to Lean

Table 1 sets the reading card of Section 1.4 beside its Lean spelling.

Section 1.4 read $a : A$ as $a \in A$ for the finite cell types of this chapter. Lean is given that reading by a list of the values:

```
class Listed (A : Type) where
  elems    : List A                -- the values of A
  complete : ∀ a, a ∈ elems        -- and every value is among them
```

With the list, `decide` checks a statement about every value of the type by running through the values. The file gives such lists for the cells of rods, the tags, and pairs and tagged pairs of listed types.

Book	Lean	Read it as
$a : A$	`a : A`	a is a value of A
$A \to B, \quad a \mapsto b$	`A → B, fun a => b`	functions
$\Sigma_{i:I} B(i), \quad (i, b)$	`(i : I) × B i, ⟨i, b⟩`	a tagged choice
$\Pi_{i:I} B(i)$	`(i : I) → B i`	a complete assignment
data $\Longrightarrow$ $C(\text{data}) : T$	`def C (data) : T := ...`	a formation line
introduction rules	`inductive T where | c : ...`	one line per rule
elimination rule	a definition by cases	one case per rule
$a \equiv b$	`rfl : a = b`	equal by computation
$\mathrm{Id}_A(a, b)$	`a = b`	the identity type
$\{s \mid \mathsf{Legal}(s)\}$	`{ s // Legal s }`	a table with its legality

Table 1: The reading card in Lean. Lean writes a tagged choice with $\times$ because, over a constant family, it is the product (Section 1.4).

Rods: their rules and their laws

Section 1.11 gave rods two introduction rules. In Lean they are an inductive type, one line for each rule, and uniform substitution (Section 1.2) is a third rule. The tags of a gluing are a type of two values.

```
inductive Tag | L | R
```

```
inductive OrderedForm where
  | rod   : Nat → OrderedForm
  | glue  : OrderedForm → OrderedForm → OrderedForm
  | subst : OrderedForm → OrderedForm → OrderedForm
```

The elimination rule is definition by cases: say what a definition gives for each rule, given what it gives for the rods the rule takes. The cells are defined so. A cell of R_n is a number below n, a cell of $\mathsf{Glue}(P, Q)$ is a tag with a cell of the rod the tag names, and a cell of $\mathsf{Substitute}(P, Q)$ is a pair.

```
def cells : OrderedForm → Type
  | rod n      => Fin n
  | glue P Q   => (t : Tag) × match t with
                              | .L => cells P
                              | .R => cells Q
  | subst P Q => cells P × cells Q
```

Size, position and order are defined by the same cases. The second and third lines of `pos` are the position equations of Section 1.2, and the last line is an instance of the computation rules of Section 1.11: `rfl` checks it for every P and Q, with no induction.

```
def size : OrderedForm → Nat
  | rod n      => n
  | glue P Q   => size P + size Q
  | subst P Q => size P * size Q
def pos : (X : OrderedForm) → X.cells → Nat
  | rod _,      i        => i
  | glue P _,   ⟨.L, p⟩  => pos P p
  | glue P Q,   ⟨.R, q⟩  => size P + pos Q q
  | subst P Q, (p, q)    => size Q * pos P p + pos Q q
def before (X : OrderedForm) (a b : X.cells) : Prop := X.pos a < X.pos b
example (P Q : OrderedForm) : (glue P Q).size = P.size + Q.size := rfl
```

Proposition 1.31 says that the order on the cells of every rod is linear. In Lean two cells with one position are one cell, and linearity follows from the order of the numbers:

```
theorem pos_inj : ∀ (X : OrderedForm) (a b : X.cells),
    X.pos a = X.pos b → a = b := · ·
theorem linear (X : OrderedForm) (a b : X.cells) :
    a ≠ b → X.before a b ∨ X.before b a := fun ne => · ·
```

The proof of `pos_inj` takes the rules in pairs, as the proof of proposition 1.31 did. For substitution it divides a position by the size of the inner rod to recover the outer cell.

Witnesses, and three equalities in $2 + 3 = 5$

A perfect alignment is a structure with five fields: a map, a map back, two equations saying that each undoes the other, and the law that the order is kept. The last three fields are proofs, so a value must supply them.

```
structure Align (P Q : OrderedForm) where
  map        : P.cells → Q.cells
  back       : Q.cells → P.cells
  back_map   : ∀ x, back (map x) = x
  map_back   : ∀ y, map (back y) = y
  keeps_order : ∀ x y, P.before x y → Q.before (map x) (map y)
```

The alignment of Figure 1.9 is written with its five arrows, in two lines for the two tags. `decide` checks the three laws on the five cells and on the twenty-five pairs of cells. A wrong arrow would fail a check, and Lean would refuse the value.

```
def α : Align (glue (rod 2) (rod 3)) (rod 5) where
  map
    | ⟨.L, i⟩ => ⟨i, by omega⟩          -- L0↦0, L1↦1
    | ⟨.R, j⟩ => ⟨2 + j, by omega⟩     -- R0↦2, R1↦3, R2↦4
  back i := if h : i < 2 then ⟨.L, ⟨i, h⟩⟩
            else ⟨.R, ⟨i - 2, by omega⟩⟩
  back_map    := by decide
  map_back    := by decide
  keeps_order := by decide
```

The three equalities of Section 1.11 are three declarations:

```
example : 2 + 3 = 5 := rfl                          -- a computation
def isGlued : OrderedForm→Prop
  | glue _ _ => True
  | _        => False
example : glue (rod 2) (rod 3) ≠ rod 5 :=           -- an empty type
  fun h => (h ▶ trivial : isGlued (rod 5))
example : Align (glue (rod 2) (rod 3)) (rod 5) := α  -- one value
```

The second is the test of Section 1.11. `isGlued` is true of every glued rod and false of every R_n. A proof `h` that the glued rod equals R_5 would turn `trivial`, the proof that the glued rod is glued, into a proof that R_5 is glued, which is `False`. So there is no such `h`.

An empty type is stated as a function into `False`: from a value it would return a proof of what has none. The rods R_2 and R_3 have no alignment (Section 1.4). The map back would send three cells into two, two of them would meet, and the map would then send one cell to two different cells. The order witness of Section 1.3 is a tagged choice whose tag is a rod: a residual C, with an alignment of $\mathsf{Glue}(R_m, C)$ with R_n. The evidence that $3 \le 5$ has the residual R_2, and its alignment is written as α was.

```
theorem no_align (a : Align (rod 2) (rod 3)) : False := ··
def Fits (m n : Nat) : Type :=
  (C : OrderedForm) × Align (glue (rod m) C) (rod n)
def threeInFive : Fits 3 5 :=
  ⟨rod 2, ··⟩
```

Laws as scripts

Each move in the table of laws of Section 1.3 is a function on cells. The exchange swaps the tags. The transpose swaps the coordinates, and transposing back is the identity by computation. The distributive

cut sends a cell of the substituted segmented rod to the block its tag names, in the same row. Regrouping is written as the exchange is. Lean refuses a definition by cases that leaves a case out, so every move is total.

```
def exchange (P Q : OrderedForm) :
    (glue P Q).cells → (glue Q P).cells
  | ⟨.L, p⟩ => ⟨.R, p⟩
  | ⟨.R, q⟩ => ⟨.L, q⟩
def transpose (m n : Nat) :
    (subst (rod m) (rod n)).cells → (subst (rod n) (rod m)).cells
  | (i, j) => (j, i)
example (m n : Nat) : transpose n m ∘ transpose m n = id := rfl
def cut (P Q₁ Q₂ : OrderedForm) :
    (subst P (glue Q₁ Q₂)).cells → (glue (subst P Q₁) (subst P Q₂)).cells
  | (p, ⟨.L, q⟩) => ⟨.L, (p, q)⟩
  | (p, ⟨.R, q⟩) => ⟨.R, (p, q)⟩
```

A tagged choice and a complete assignment

The family F of Section 1.4 is a function from tags to rods. A value of the tagged choice is a tag with a cell of the rod it names, and the list of values has five entries, none repeated:

```
def F : Tag → OrderedForm
  | .L => rod 2
  | .R => rod 3
example : (t : Tag) × (F t).cells := ⟨.R, 1⟩          -- the cell R1
def values (A : Type) [Listed A] : List A := Listed.elems
example : (values ((t : Tag) × (F t).cells)).length = 5 := rfl
example : (values ((t : Tag) × (F t).cells)).Nodup := by decide
```

Lean refuses the pair $(L, 2)$:

```
example : (t : Tag) × (F t).cells := ⟨.L, ⟨2, by decide⟩⟩
```

Lean refuses it:

```
Tactic 'decide' proved that the proposition
  2 < 2
is false
```

Written $\langle 2, h\rangle$, the cell carries a proof h that $2 < 2$, and there is none. A bare numeral would not be checked: Lean reads a numeral in `Fin n` modulo n, so that $\langle L, 2\rangle$ would be the cell $(L, 0)$.

A complete assignment is a function defined at both tags. It corresponds to the pairs of a row value and a column value (Section 1.4) by two functions, and the two examples after them are the round trips. The first holds by computation. The second compares functions and uses `funext`: functions that agree at every input are equal. Lean proves this principle from its quotient types; an intensional type theory without them must assume it.

```
def f : (t : Tag) → (F t).cells                    -- [L↦0, R↦1]
  | .L => 0
  | .R => 1
def toPair (g : (t : Tag) → (F t).cells) : Fin 2 × Fin 3 := (g .L, g .R)
def ofPair : Fin 2 × Fin 3 → (t : Tag) → (F t).cells
  | (i, _), .L => i
  | (_, j), .R => j
example (x : Fin 2 × Fin 3) : toPair (ofPair x) = x := rfl
example (g : (t : Tag) → (F t).cells) : ofPair (toPair g) = g :=
  funext fun t => by cases t <;> rfl
example : (values (Fin 2 × Fin 3)).length = 6 := rfl
```

Formation lines

A formation line becomes the head of a declaration: its inputs are the parameters, and the constructed type follows the colon. On cells, the assembly of Section 1.4 is the tagged choice itself. The code and the history of a form (Figure 1.21) are a tagged choice whose tag is the

code. `History k` holds the construction trees of k cells: an inductive family, with one type for each code. The two trees of Figure 1.22 are two values over the code 3.

```
def assemble (I : Type) (F : I → Type) : Type := (i : I) × F i
inductive History : Nat → Type
  | atom  : Char → History 1
  | glue  : History m → History n → History (m + n)
  | subst : History m → History n → History (m * n)
def AFForm : Type := (k : Nat) × History k
def treeA : AFForm :=
  ⟨3, .glue (.atom 'a')
        (.subst (.glue (.atom 'b') (.atom 'c')) (.atom 'd'))⟩
def treeB : AFForm :=
  ⟨3, .glue (.atom 'a') (.glue (.atom 'd') (.atom 'd'))⟩
```

A tree of four cells is a history of the four-cell code and of no other:

```
def ab : History 2 := .glue (.atom 'a') (.atom 'b')
example : AFForm := ⟨3, .glue ab ab⟩
```

Lean refuses it:

```
Application type mismatch: The argument
  ab.glue ab
has type
  History (2 + 2)
but is expected to have type
  History 3
in the application
  ⟨3, ab.glue ab⟩
```

The base cut of a pointed wheel is a cell of the wheel's own form, so the type of the second field mentions the first. The arc state and the gapped state of Figure 1.15 are two tables with one tally. Normalization takes its evidence as an input, and so cannot be applied to R_2 and R_3.

```
structure Wheel where
  F    : OrderedForm
  base : F.cells                          -- the cut: a cell of F itself
def eight : Wheel := ⟨rod 8, 0⟩
def tally (s : Fin 8→Bool) : Nat :=
  ((List.finRange 8).filter (s ·)).length
def arc    : Fin 8→Bool := fun i => i.val ∈ [0, 1, 2]
def gapped : Fin 8→Bool := fun i => i.val ∈ [1, 4, 6]
example : tally arc = 3 ∧ tally gapped = 3 ∧ arc ≠ gapped := by decide
def normalize (X : OrderedForm) (n : Nat) (_ : Align X (rod n)) :
    OrderedForm :=
  rod n
example : OrderedForm := normalize (glue (rod 2) (rod 3)) 5 α
```

Time

A finite temporal support has positions, a successor defined on some of them, and the proof that the successor is injective. The line and the cycle supply that proof once for every n, and a clock is a pointed cycle.

```
structure TimeForm where
  n    : Nat
  next : Fin n→Option (Fin n)             -- none: no successor
  inj  : ∀ s t u, next s = some u→next t = some u→s = t
def LinearTime (n : Nat) : TimeForm where
  n := n
  next t := if h : t + 1 < n then some ⟨t + 1, h⟩ else none
  inj s t u h₁ h₂ := · ·
def CycleTime (n : Nat) : TimeForm where
  n := n
  next t := some ⟨(t + 1) % n, Nat.mod_lt _ (by have := t.isLt; omega)⟩
  inj s t u h₁ h₂ := · ·
structure PointedTimeForm where
  T  : TimeForm
  t₀ : Fin T.n
def PointTime (T : TimeForm) (t₀ : Fin T.n) : PointedTimeForm := ⟨T, t₀⟩
```

```
def forgetPoint (P : PointedTimeForm) : TimeForm := P.T
def Clock (L : Nat) (h : 0 < L) : PointedTimeForm :=
  PointTime (CycleTime L) ⟨0, h⟩
```

A rectangular tapestry (definition 1.5) keeps its support, its ordered rows, its temporal palette and one thread for every row. Lean numbers the rows of the tetractys palette from 0, so row r has $r + 1$ phases. Each thread of Figure 1.25 reads a tick modulo the number of phases of its row, and the names $1, \ldots, 10$ are computed for the reader. The two outlined columns, the distinctness of the twelve columns and the ten values of the palette are checked:

```
structure RectTapestry where
  T : TimeForm
  R : OrderedForm                          -- the rows, in order
  V : R.cells→Type                         -- the temporal palette
  e : (r : R.cells)→Fin T.n→V r            -- one thread for every row
def thread (r : Fin 4) (t : Fin 12) : Fin (r + 1) :=
  ⟨t % (r + 1), Nat.mod_lt _ (by omega)⟩
def recurrence : RectTapestry :=
  ⟨CycleTime 12, rod 4, fun r => Fin (r + 1), thread⟩
def name (r : Fin 4) (j : Fin (r + 1)) : Nat := 1 + (r + 1) * r / 2 + j
def column (t : Fin 12) : List Nat :=
  (List.finRange 4).map fun r => name r (thread r t)
example : column 0 = [1, 2, 4, 7] ∧ column 5 = [1, 3, 6, 8] := by decide
example : ((List.finRange 12).map column).Nodup := by decide
example : (values ((r : Fin 4) × Fin (r + 1))).length = 10 := rfl
```

A laying sends the number of phases of a row and a tick to a phase. The period of a row is the least step after which it repeats, and the spectra of proposition 1.10 are computed:

```
def Laying := Nat→Nat→Nat          -- phases k and tick t↦a phase
def layRec : Laying := fun k t => t % k
def layDiv : Laying := fun k t => k * t / 12
def period (row : Nat→Nat) : Nat :=
  ((List.range 12).map (· + 1)).find? (fun d => 12 % d == 0 &&
```

```
      (List.range 12).all fun t => row ((t + d) % 12) == row t)
    |>.getD 12
def spectrum (ℓ : Laying) : List Nat :=
  [1, 2, 3, 4].map fun k => period (ℓ k)
example : spectrum layRec = [1, 2, 3, 4]    := by decide
example : spectrum layDiv = [1, 12, 12, 12] := by decide
```

An occurrence history (definition 1.6) is a path in a transition graph: an inductive family indexed by its two ends and its number of transitions. A tick realization (definition 1.7) keeps its certificate as a field, so no value can be formed without it. Every certified record of Section 1.5 has this shape.

```
structure Graph where
  State : Type
  Move  : State → State → Type
inductive Path (G : Graph) : G.State → G.State → Nat → Type
  | stay : Path G x x 0
  | step : G.Move x y → Path G y z H → Path G x z (H + 1)
structure TickRealization (G : Graph) (T : PointedTimeForm) where
  q     : G.State → Fin T.T.n                    -- phase projection
  x₀    : G.State
  κ     : {x y : G.State} → G.Move x y → Bool     -- registers a tick?
  start : q x₀ = T.t₀
  law   : ∀ {x y} (a : G.Move x y),
            if κ a then T.T.next (q x) = some (q y) else q y = q x
```

The digits of a cyclarchy (definition 1.8) are a complete assignment over a family that varies, one digit for each radix, written as nested pairs. Adding one carries into the next digit, and flattening makes the cyclarchy $(3, 4)$ a clock of twelve:

```
def Digits : List Nat → Type
  | []      => Unit
  | N :: Ns => Fin N × Digits Ns
def addCarry : (Ns : List Nat) → Digits Ns → Bool → Digits Ns
  | [],      (),     _ => ()
  | N :: Ns, (a, r), c =>
```

```
      if c then
        if h : a + 1 < N then (⟨a + 1, h⟩, r)
        else (⟨0, by omega⟩, addCarry Ns r true)   -- the carry
      else (a, r)
def inc (Ns : List Nat) (d : Digits Ns) : Digits Ns := addCarry Ns d true
def flatten : (Ns : List Nat) → Digits Ns → Nat
  | [],      ()     => 0
  | N :: Ns, (a, r) => a + N * flatten Ns r
example : inc [3, 4] (⟨2, by omega⟩, ⟨0, by omega⟩, ())
          = (⟨0, by omega⟩, ⟨1, by omega⟩, ()) := by decide
example : ∀ d : Digits [3, 4],
    flatten [3, 4] (inc [3, 4] d) = (flatten [3, 4] d + 1) % 12 := by
  decide
```

The covering and grouping equations of Section 1.5 are two statements about a laid row, and the division laying fails the first:

```
def covering (L k : Nat) (c : Nat → Nat) : Prop :=
  ∀ t < L, c ((t + 1) % L) = (c t + 1) % k
def grouping (L k : Nat) (p : Nat → Nat) : Prop :=
  ∀ t < L, p ((t + L / k) % L) = (p t + 1) % k
example : covering 12 4 (layRec 4) ∧ grouping 12 4 (layDiv 4) := by decide
example : ¬ covering 12 4 (layDiv 4) := by decide
```

The calibration of Figure 1.38, the gear phases, the event tapestry and the duration record are declared in the same way, and the closing table of this part names them. The calibration record carries as its certificate a proof that every pair of readings meets the law, and the file computes from it the elapsed interval $[2999, 3001]U$.

Meaning

An interpretation profile declares its endpoints, its realizers and its law. An interpretation witness (definition 1.9) is a realizer with its certificate. The semantic fibre is a tagged choice over the profiles in use, and the selected layer a further one.

```
structure Profile (Src : Type) where
  End       : Type                          -- what a meaning may produce
  Realizer  : Type                          -- the rules that may produce it
  produces  : Realizer→Src→End→Prop
  law       : Realizer→Prop
structure IntWit (C : Profile Src) (F : Src) (θ : C.End) where
  realizer : C.Realizer
  cert     : C.law realizer ∧ C.produces realizer F θ
def Sem (Γ : I→Profile Src) (F : Src) : Type :=
  (i : I) × (θ : (Γ i).End) × IntWit (Γ i) F θ
def PointedInt (Γ : I→Profile Src) : Type := (F : Src) × Sem Γ F
```

The two disciplines of the game of Section 1.7 are two profiles. A law asks for its equation and for the phase typing of the rows; a profile asks that the frame be what the laying produces, with the declared spectrum. The two moves are two witnesses, and the moves side by side are two values of the fibre. The move that was not made is an empty type: no laying at all strings the palette into the division frame under the law of the clock. The frame shows the row of two unchanged from tick 0 to tick 1, and the covering equation requires a change.

```
def Palette := List Nat                      -- the row sizes of the theme
def P : Palette := [1, 2, 3, 4]
def Strung := Fin 4→Fin 12→Nat               -- four wires, twelve rungs
def lay (ℓ : Laying) (F : Palette) : Strung := fun r t => ℓ (F.getD r 0) t
def clockLaw (ℓ : Laying) : Prop :=
  ∀ k ∈ [1, 2, 3, 4], covering 12 k (ℓ k) ∧ ∀ t < 12, ℓ k t < k
def stringLaw (ℓ : Laying) : Prop :=
  ∀ k ∈ [1, 2, 3, 4], grouping 12 k (ℓ k) ∧ ∀ t < 12, ℓ k t < k
def profile (law : Laying→Prop) (spec : List Nat) :
    Profile Palette where
  End       := Strung
  Realizer  := Laying
  produces ℓ F θ := θ = lay ℓ F ∧ spectrum ℓ = spec
  law       := law
def Crec := profile clockLaw  [1, 2, 3, 4]
```

```
def Cdiv := profile stringLaw [1, 12, 12, 12]
def wRec : IntWit Crec P (lay layRec P) :=
  ⟨layRec, (by decide : clockLaw layRec), rfl, by decide⟩
def wDiv : IntWit Cdiv P (lay layDiv P) :=
  ⟨layDiv, (by decide : stringLaw layDiv), rfl, by decide⟩
inductive Discipline | clock | string
def Γ : Discipline→Profile Palette
  | .clock  => Crec
  | .string => Cdiv
def moves : List (Sem Γ P) := [⟨.clock, _, wRec⟩, ⟨.string, _, wDiv⟩]
theorem no_move : IntWit Crec P (lay layDiv P)→False := · ·
```

Spaces and states

A finite cell pattern (definition 1.11) declares cells, their dimensions, the immediate faces and a face-closed boundary, with the two laws as fields. SquareGrid(m, n) gives the cells of each dimension.

```
structure Canvas where
  Cell     : Type
  dim      : Cell→Nat
  face     : Cell→Cell→Prop        -- a is an immediate face of b
  raises   : ∀ a b, face a b→dim b = dim a + 1
  boundary : Cell→Prop              -- the declared outer boundary
  closed   : ∀ a b, face a b→boundary b→boundary a
inductive Dim | vertex | edge | face
def SquareGrid (m n : Nat) : Dim→Type
  | .vertex => Fin (m + 1) × Fin (n + 1)
  | .edge   => Fin m × Fin (n + 1) ⊕ Fin (m + 1) × Fin n
  | .face   => Fin m × Fin n
```

Listing the cells of the board gives the counts of Section 1.8. The colouring rule, the shared edge and the domino of Figure 1.48 follow. The placement at $d6$ and $e7$ has no value, since its field would prove that `false = true`; `nofun`, the function with no cases, shows it.

```
def count (A : Type) [Listed A] : Nat := (values A).length
example : count (SquareGrid 8 8 .vertex) = 81  := rfl
example : count (SquareGrid 8 8 .edge)   = 144 := rfl
example : count (SquareGrid 8 8 .face)   = 64  := rfl
def Square := Fin 8 × Fin 8                -- (file, rank): a1 is (0, 0)
def dark (s : Square) : Bool := (s.1.val + s.2.val) % 2 == 0
def shareEdge (s t : Square) : Bool :=
  (s.1 == t.1 && (s.2.val + 1 == t.2.val || t.2.val + 1 == s.2.val)) ||
  (s.2 == t.2 && (s.1.val + 1 == t.1.val || t.1.val + 1 == s.1.val))
structure Domino (a b : Square) : Prop where
  link : shareEdge a b = true
example : Domino (6, 6) (7, 6) := ⟨rfl⟩          -- g7 and h7
example : Domino (3, 5) (4, 6)→False := nofun    -- d6 and e7
```

The state space is a subtype. Lean treats any two proofs of one proposition as equal, so two states with one table are equal by computation. This is legality read as a property (Section 1.11): Lean counts tables, not proofs.

```
def Table := Square→Fin 13              -- six white, six black, empty
def State (Legal : Table→Prop) : Type := { s : Table // Legal s }
example (Legal : Table→Prop) (s : Table) (p q : Legal s) :
    (⟨s, p⟩ : State Legal) = ⟨s, q⟩ := rfl
```

Scripts and roles

A script retains its operations as well as its action. The transpose followed by itself acts as the identity and records two operations, as Section 1.3 said of the two-step script. Positions with White to play and positions with Black to play are two types, and a White turn goes from the first to the second (Figure 1.57).

```
structure Script (A B : Type) where
  ops : List String                    -- the operations it retains
  act : A→B                            -- its action on states
def Script.comp (g : Script B C) (f : Script A B) : Script A C :=
  ⟨f.ops ++ g.ops, g.act ∘ f.act⟩
def τ (m n : Nat) : Script (Fin m × Fin n) (Fin n × Fin m) :=
  ⟨["transpose"], fun (i, j) => (j, i)⟩
example : ((τ 3 2).comp (τ 2 3)).act = id := rfl
example : ((τ 3 2).comp (τ 2 3)).ops = ["transpose", "transpose"] := rfl
structure WhiteToPlay where board : Table
structure BlackToPlay where board : Table
def turnW (move : Table→Table) : Script WhiteToPlay BlackToPlay :=
  ⟨["White's move"], fun s => ⟨move s.board⟩⟩
```

Two White turns do not compose:

```
example (m : Table→Table) : Script WhiteToPlay BlackToPlay :=
  (turnW m).comp (turnW m)
```

Lean refuses it:

```
Application type mismatch: The last
  turnW m
argument has type
  Script WhiteToPlay BlackToPlay
but is expected to have type
  Script WhiteToPlay WhiteToPlay
in the application
  (turnW m).comp (turnW m)
```

`#eval` prints a value for the reader, and no later declaration can refer to what it printed. A definition is a value that later declarations may use.

```
def lit : Fin 8→Bool := arc          -- derived data: a rule may read it
#eval tally lit                      -- prints 3, a reading: no rule can
```

What Lean does not see

Lean checks that each value has its type. As Section 1.11 said of the rules, it does not say what a value is for. The material balance of a position could be computed by one definition and consumed by the next, and Lean would accept both. The rule of Section 1.9 that nothing may depend on a reading is kept by whoever writes the program, by leaving readings in `#eval`.

Nor does Lean see pictures. Section 1.4 specified a structure by a picture and stated its laws in the text. In Lean the inductive type is the specification, and a law the picture shows is a theorem, as `pos_inj` is for the row of cells.

Every formation line in Lean

Each formation line of this chapter, and the declaration of the file that writes it:

Formation line	Lean
Glue(P, Q), R_n, Substitute(P, Q)	`OrderedForm`, three rules
Assemble$(I; F)$, Index(n), $\varnothing$, an atom	`assemble`, `Fin n`, `Fin 0`, `Fin 1`
code k and history h : History(k)	`AFForm`, `History`
Wheel$_n(F; b)$, Normalize$(X; \alpha)$	`Wheel`, `normalize`
LinearTime(n), CycleTime(n), PointTime$(T; t_0)$	`LinearTime`, `CycleTime`, `PointTime`
toClock$(W; i)$, SampleClock	`toClock`, `SampleClock`
RectTapestry$(T; R, V, e)$	`RectTapestry`
OccurrenceHistory$(G; \gamma)$	`Path`

Formation line	Lean
$\mathsf{TickRealization}(G, T)$	`TickRealization`
$\mathsf{Cyclarchy}(N)$	`Digits`, `inc`, `flatten`
$\mathsf{EventTapestry}(T; D)$	`EventData`
$\mathsf{DurationRecord}(\gamma; U, d)$	`DurationRecord`, `D`
$\mathsf{EventAssociation}(E, \gamma)$	`EventAssociation`
$\mathsf{CalibrationRecord}(R; \rho, \epsilon, c)$, $\mathsf{Elapsed}(w; e, f)$	`CalibrationRecord`, `elapsed`
$\mathsf{IntWit}_{\mathcal{C}}(F, \theta)$, $\mathsf{Sem}_{\Gamma}(F)$, $\mathsf{PointedInt}_{\Gamma}$	`IntWit`, `Sem`, `PointedInt`
$\mathsf{SquareGrid}(m, n)$, $\mathsf{Canvas}(F; c)$	`SquareGrid`, `Canvas`
$\mathsf{AttachCell}(X, c; w)$	`attachCell`, `PatchWitness`
$\mathsf{State}(A)$, a frame and its instances	`State`, `Frame`, `Frame.Instance`
$\mathsf{Refine}(X; \rho)$	`RefinementData`
$g \circ f : \mathsf{Script}(A, C)$	`Script.comp`

The file also counts the eight symmetries of the square of Section 1.8 from its incidence.

The same constructions in the library

Part III presents the repository's library, which writes the book in Lean with Mathlib. Several constructions of this part appear there, some in a different form. The names of the library are in the namespace `Artform`.

This part	The library	Difference
`Listed`	Mathlib's `Fintype`	Mathlib's class of finite types
`OrderedForm`, `cells`, `before`	`OrderedForm`, `OrderedForm.cells`, `OrderedForm.before`	two rules, as in Section 1.11; the order is defined by cases, not by position
`pos_inj`, `linear`	`rodLaws`	proposition 1.31
`Align`	`Align`	an order-preserving bijection of finite linear orders
`TimeForm`, `CycleTime`	`TimeForm`, `CycleTime`	
`RectTapestry`	`RectTapestry`	the support is a parameter
`Laying`, `layRec`, `layDiv`	`Laying`, `layRec`, `layDiv`	
`covering`, `grouping`	`CoveringEq`, `GroupingEq`	
`period`, `spectrum`	`RectTapestry.leastPeriod`, `RectTapestry.tempSpec`	
`Profile`, `IntWit`, `Sem`, `PointedInt`	`IntProfile`, `IntWit`, `Sem`, `PointedInt`	definition 1.9
`Palette`, `P`, `Crec`, `Cdiv`, `wRec`, `wDiv`	`TemporalPalette`, `tetractys`, `Crec`, `Cdiv`, `wRec`, `wDiv`	proposition 1.10 is proved

This part	The library	Difference
`Canvas`	`Canvas`	dimension two, with a stored orientation (definition 3.1)
`State`, `Frame`	`Presentation`, `Presentation.Table`	
`Script`, `Script.comp`	`ArtSyn`	Mathlib's free category on the declared operations

Connections and sources

Lean 4 is described by de Moura and Ullrich. Its type theory goes back to the calculus of constructions of Coquand and Huet, with the inductive types of Coquand and Paulin; inductive families, such as the histories and paths of this part, are Dybjer's. *Sources:* [6, 4, 5, 7].

Part III. The textbook's theorems in Lean

The repository that holds this guide also holds the numbered statements of the book in Lean, with proofs that Lean checks. Part II wrote the constructions of Chapter 1 in core Lean, in one file. The repository's library is written with Mathlib, Lean's library of mathematics, and follows the whole book. This part says what the repository proves and how to build it, introduces the parts of Mathlib that the library uses, and shows how a statement of the book is written in Lean. It then reads four statements closely, from the book's words to their proofs, and ends with the checks the repository makes on itself.

What the repository proves

The repository gives each of the book's 103 numbered statements one of five states:

- *planned*: no Lean statement has been written yet;
- *stated*: a Lean statement has been written, and it waits for the author's review, or for a proof;

- *proved*: the author has approved the Lean statement, and Lean has checked a proof of it;
- *partial*: as proved, for a narrower statement, with the reason recorded;
- *informal*: not to be written in Lean, by decision, with the reason recorded.

A statement is proved in this sense when three things hold. The library has a Lean proposition tied to the statement's label in the book. The author has read that proposition beside the book and approved it as saying what the book says. And Lean has checked a proof of it that uses no axiom beyond the three that the last section of this part describes.

Lean checks the third point, and only the third: that the proof proves the Lean proposition. It cannot check the second, which compares a text in mathematical English with a text in Lean. Three things help the author, and any reader, to judge it. Every Lean statement has a *fidelity note*, which lists each way it reads the book: a hypothesis made explicit, a case generalized, a choice between two readings. Every proof file ends with an example on the book's running example in which the hypotheses hold, so that no statement is true merely because its hypotheses cannot be met. And an approval records two hashes, of the statement's text and of its exact terms. If the statement, or any definition it uses, changes later, the statement goes back for review.

When this guide was built, 0 statements were proved, 80 were stated and 23 were planned. The appendix lists all of them. The file `textbook/STATUS.md` of the repository is always current.

Getting and building it

The repository is on GitHub. To build it, install VS Code and its Lean 4 extension, as in Part I. The extension also installs `elan`, which installs the version of Lean that a project asks for, and `lake`, Lean's build tool. Then, in a terminal:

```
git clone https://github.com/skrivov/artform-calculus-proofs
cd artform-calculus-proofs
lake exe cache get
lake build
```

The first two lines copy the repository and enter it. The file `lean-toolchain` names the version of Lean, 4.34.1, and `elan` installs it the first time `lake` runs. `lake exe cache get` fetches Mathlib and downloads its compiled files, about 450 MB, instead of compiling Mathlib, which would take hours. Unpacked, they take about 8 GB. `lake build` then compiles the library itself in a few minutes. It prints a line for each file, and an error for a file that Lean refuses.

In VS Code, open the folder of the repository. The extension checks each file as it is opened and shows Lean's answers in the infoview, as in Part I. The Lean playground in the browser has Mathlib but not this library, so the files of the repository cannot be tried there. The files of Parts I and II can.

From core Lean to Mathlib

Parts I and II used Lean's core language alone. The library uses Mathlib, a library of mathematics written in Lean and kept by its community [8]. It has the integers modulo n, finite sums, groups, linear maps, measures and categories, each with its theorems. A file

asks for a part of Mathlib with `import`. The listings of this section are cut from the file `RepositoryTour.lean`, which imports the whole library. As in Parts I and II, Lean checks every answer they show.

Finite types. Part II gave each finite type a list of its values, the class `Listed`, so that `decide` could run through them. Mathlib's class for the same purpose is `Fintype`, and `Fintype.card` counts the values. The library gives the cells of every rod an instance of it, so a glued rod can be counted:

```
#eval Fintype.card (Fin 5)                                    -- 5
#eval Fintype.card (Fin 2 × Fin 3)                            -- 6
#eval Fintype.card (OrderedForm.glue (.rod 2) (.rod 3)).cells -- 5
example : ∀ i : Fin 10, i.val * i.val % 4 ≠ 2 := by decide
```

The last line is the check of Part I on squares and remainders. `decide` works as it did there, with Mathlib's instances.

Finite sums. A `Finset` is a finite set of values of a type, and `Σ k ∈ s, f k` is the sum of `f` over the set `s`. `Finset.range 5` is $\{0, 1, 2, 3, 4\}$, so the first sum counts the ten points of the tetractys again. The last example proves Gauss's formula of Part I for Mathlib's sums. `Finset.sum_range_succ` is a theorem of Mathlib that splits off the last term of a sum over a range, and `ring` proves the polynomial identity that is left.

```
#eval Σ k ∈ Finset.range 5, k  -- 10
example : Σ k ∈ Finset.range 5, k = 10 := by decide
example (n : ℕ) : 2 * Σ k ∈ Finset.range (n + 1), k = n * (n + 1) := by
  induction n with
  | zero => rfl
  | succ n ih => rw [Finset.sum_range_succ, mul_add, ih]; ring
```

The integers modulo n. `ZMod n` is the integers modulo n, with their addition and multiplication. The toy electrodynamics of Chapter 6 counts modulo 8: there $5 + 4$ is 1, and $3 - 5$ is 6. `ZMod n` is finite, so `decide` checks a statement for all its values. Eight equal steps bring every value back, and a number modulo 3, the values of the square's law (Figure 5.1), is 0, 1 or 2:

```
#eval (5 : ZMod 8) + 4  -- 1
#eval (3 : ZMod 8) - 5  -- 6
example : ∀ a : ZMod 8, a + a + a + a + a + a + a + a = 0 := by decide
example : ∀ c : ZMod 3, c = 0 ∨ c = 1 ∨ c = 2 := by decide
```

Permutations. A bijection of a type to itself is a permutation, `Equiv.Perm`. Permutations form a group, written as a product, and `f * g` is $f \circ g$: first g, then f, as the last line checks by computation. `finRotate 4` moves each of the numbers $0, 1, 2$ on by one and 3 to 0, as the tick of the square moves each value one corner on. Its fourth power is the identity and its square is not:

```
#eval finRotate 4 3  -- 0
example : finRotate 4 ^ 4 = 1 := by decide
example : finRotate 4 ^ 2 ≠ 1 := by decide
example (f g : Equiv.Perm (Fin 4)) (x : Fin 4) : (f * g) x = f (g x) := rfl
```

Additive maps. A tally of Chapter 5 is additive: $T(p + q) = T(p) + T(q)$. Mathlib's type of additive maps from `A` to `B` is `A →+ B`: a function together with the proof that it is additive. A theorem about all additive maps, such as `map_add`, then applies to every tally. The library writes the law of the square and its total T this way. The tick moves each value one corner on, and the total of $(1, 2, 0, 1)$ is 4, which is 1 modulo 3:

```
#eval squareLaw.tick ![1, 2, 0, 1]  -- ![1, 1, 2, 0]
#eval squareTotal ![1, 2, 0, 1]     -- 1
example (p q : Fin 4→ZMod 3) :
    squareTotal (p + q) = squareTotal p + squareTotal q :=
  map_add squareTotal p q
```

Finding a theorem. Mathlib has about 180 000 theorems, and the first question in a proof is often whether the step needed is one of them. `exact?` searches the theorems in scope for one that closes the goal, and prints the call it found:

```
example (a b : ℕ) : a + b = b + a := by exact?
```

Lean replies:

```
Try this:
  [apply] exact Nat.add_comm a b
```

Loogle (https://loogle.lean-lang.org) searches Mathlib by name and by the shape of a statement. The documentation of Mathlib (https://leanprover-community.github.io/mathlib4_docs) lists every definition and theorem, with its source.

How the book is written in Lean

The library has three kinds of declaration:

- A definition of the book is a definition of Lean in the namespace `Artform`: a structure, an inductive type or a function. The rods of Chapter 1 are one, the canvas of definition 3.1 another.
- A statement of the book is a definition of a proposition, `def ... : Prop`, in the namespace `Artform.Statement`. It says what the book says, and proves nothing.

- Its proof is a theorem whose type is exactly that proposition.

The attribute `@[book "label"]` ties a declaration to the statement or definition of the book that carries that label. Labels are used, not numbers, because the numbers change between editions of the book and the labels do not. The command `#book` lists the declarations tied to a label:

```
#book "thm:noether"
```

Lean replies:

```
Artform.Statement.noether : Prop
Artform.noether : Statement.noether
```

The statement of Noether's theorem is `Artform.Statement.noether`, a proposition, and its proof is the theorem `Artform.noether`, whose type is that proposition. Keeping the two apart lets the author review the statement without reading the proof, and lets a proof be replaced without touching what it proves.

Each chapter of the book has a folder of the library, `Ch01` to `Ch13`. Its `Defs` folder holds the chapter's definitions, `Statements.lean` or the `Statements` folder its statements, and the other files their proofs. The file `textbook/manifest.toml` gives every statement and definition of the book its Lean names, its status and its fidelity note. `textbook/STATUS.md` and the appendix of this guide are made from it.

Four statements read closely

The four statements below come from Chapters 1 and 5 of the book. Each is shown as the book prints it and as the library states it, with a table that reads one into the other, then with its proof or the outline

of its proof, and with its fidelity note. In the library's listings the documentation comments are left out; the files in the repository have them.

The laws of a rod

Part II built rods by three rules, substitution being the third, and ordered their cells by position. The library follows Section 1.11, which builds rods by two rules and defines their order by cases:

```
inductive Tag where
  | L
  | R
  deriving DecidableEq
inductive OrderedForm where
  | rod : ℕ→OrderedForm
  | glue : OrderedForm→OrderedForm→OrderedForm
namespace OrderedForm
@[reducible] def cells : OrderedForm→Type
  | rod n => Fin n
  | glue P Q => (t : Tag) × (match t with
      | .L => P.cells
      | .R => Q.cells)
def before : (X : OrderedForm) →X.cells→X.cells→Prop
  | rod _, i, j => i < j
  | glue P _, ⟨.L, p⟩, ⟨.L, p'⟩ => P.before p p'
  | glue _ _, ⟨.L, _⟩, ⟨.R, _⟩ => True
  | glue _ _, ⟨.R, _⟩, ⟨.L, _⟩ => False
  | glue _ Q, ⟨.R, q⟩, ⟨.R, q'⟩ => Q.before q q'
```

`cells` is the elimination rule of Section 1.11 written as a definition by cases. A cell of R_n is a number below n, and a cell of $\mathsf{Glue}(P, Q)$ is a tag with a cell of the rod that the tag names. `before` says, case by case, when one cell comes before another: in R_n by the order of the numbers, and in $\mathsf{Glue}(P, Q)$ every cell tagged L before every cell tagged R, and two cells with one tag as in P or in Q.

The attribute `@[reducible]` lets Lean see through `cells`, so that Mathlib's instances for `Fin n` serve the cells of R_n.

Proposition 1.31 The laws of a rod

For every rod X the order on $\mathsf{Cell}(X)$ is a linear order: no cell comes before itself; if a comes before b and b before c, then a comes before c; and of two different cells one comes before the other.

```
@[book "prop:rod-laws"]
def rodLaws : Prop :=
  ∀ X : OrderedForm,
    (∀ a : X.cells, ¬ X.before a a) ∧
    (∀ a b c : X.cells, X.before a b → X.before b c → X.before a c) ∧
    (∀ a b : X.cells, a ≠ b → X.before a b ∨ X.before b a)
```

Book	Lean
$\mathsf{Cell}(X)$	`X.cells`
a comes before b	`X.before a b`
no cell comes before itself	`¬ X.before a a`
of two different cells one comes before the other	`X.before a b ∨ X.before b a`, when `a ≠ b`

The proof follows the book's, by induction over the two rules:

```
@[book "prop:rod-laws"]
theorem rodLaws : Statement.rodLaws := by
  intro X
  induction X with
  | rod n =>
    exact ⟨fun a => lt_irrefl a, fun _ _ _ => lt_trans,
      fun _ _ => lt_or_gt_of_ne⟩
  | glue P Q ihP ihQ =>
```

```
obtain ⟨irrP, transP, totP⟩ := ihP
obtain ⟨irrQ, transQ, totQ⟩ := ihQ
refine ⟨?_, ?_, ?_⟩
· rintro ⟨_ | _, a⟩
  · exact irrP a
  · exact irrQ a
· rintro ⟨_ | _, a⟩ ⟨_ | _, b⟩ ⟨_ | _, c⟩ hab hbc
  · exact transP a b c hab hbc
  · exact trivial
  · exact hbc.elim
  · exact trivial
  · exact hab.elim
  · exact hab.elim
  · exact hbc.elim
  · exact transQ a b c hab hbc
· rintro ⟨_ | _, a⟩ ⟨_ | _, b⟩ hab
  · exact totP a b fun h => hab (h ▶ rfl)
  · exact Or.inl trivial
  · exact Or.inr trivial
  · exact totQ a b fun h => hab (h ▶ rfl)
```

`induction X with` splits the goal into the two rules. For a glued rod it supplies the three laws for P and for Q, which `obtain` takes apart into `irrP`, `transP`, `totP` and the same for Q. `refine ⟨?_, ?_, ?_⟩` leaves one goal for each law. Each `rintro ⟨_ | _, a⟩` introduces a cell and splits it by its tag, so that each law becomes one goal for each combination of tags: two for the first law, eight for the second and four for the third. Cells with one tag use the laws of P or of Q. A cell tagged L comes before a cell tagged R by `trivial`, and the remaining cases have `False` as a hypothesis, which `elim` turns into any goal.

Fidelity note. Rods are built by the two rules of Section 1.11, and their cells and order by its computation rules.

Reversing a tick of two steps

Proposition 5.39 Reversing a tick of two steps

Let a tick be $F = S_1 \circ S_0$, and let an involution R of the states undo each step: $R \circ S_0 \circ R = S_0^{-1}$ and $R \circ S_1 \circ R = S_1^{-1}$. Then $I_0 = R \circ S_0$ and $I_1 = S_1 \circ R$ are involutions, and each reverses the tick:

$$I_0 \circ F \circ I_0 = F^{-1} = I_1 \circ F \circ I_1.$$

The states are any type `S`, and the two steps and the involution are permutations of it, so that S_0^{-1} and S_1^{-1} exist. In the group of permutations `f * g` is $f \circ g$:

```
@[book "prop:two-step-reversal"]
def twoStepReversal : Prop :=
  ∀ (S : Type) (S₀ S₁ R : Equiv.Perm S),
    R * R = 1 → R * S₀ * R = S₀⁻¹ → R * S₁ * R = S₁⁻¹ →
      (R * S₀) * (R * S₀) = 1 ∧ (S₁ * R) * (S₁ * R) = 1 ∧
      (R * S₀) * (S₁ * S₀) * (R * S₀) = (S₁ * S₀)⁻¹ ∧
      (S₁ * R) * (S₁ * S₀) * (S₁ * R) = (S₁ * S₀)⁻¹
```

Book	Lean
$F = S_1 \circ S_0$	`S₁ * S₀`
R is an involution	`R * R = 1`
$R \circ S_0 \circ R = S_0^{-1}$	`R * S₀ * R = S₀⁻¹`
$I_0 = R \circ S_0$, $I_1 = S_1 \circ R$	`R * S₀`, `S₁ * R`
I_0 is an involution	`(R * S₀) * (R * S₀) = 1`
$I_0 \circ F \circ I_0 = F^{-1} = I_1 \circ F \circ I_1$	the last two lines

The proof is the book's chain of rewrites:

```
@[book "prop:two-step-reversal"]
theorem twoStepReversal : Statement.twoStepReversal := by
  intro S S₀ S₁ R hR h₀ h₁
  -- From `R ∘ Sᵢ ∘ R = Sᵢ⁻¹` follows `Sᵢ ∘ R ∘ Sᵢ = R`.
  have k₀ : S₀ * R * S₀ = R := by
    have h : R * (S₀ * R * S₀) = 1 := by
      rw [←mul_assoc, ←mul_assoc, h₀, inv_mul_cancel]
    rw [eq_inv_of_mul_eq_one_right h, inv_eq_of_mul_eq_one_right hR]
  have k₁ : S₁ * R * S₁ = R := by
    have h : R * (S₁ * R * S₁) = 1 := by
      rw [←mul_assoc, ←mul_assoc, h₁, inv_mul_cancel]
    rw [eq_inv_of_mul_eq_one_right h, inv_eq_of_mul_eq_one_right hR]
  -- `F⁻¹ = S₀⁻¹ ∘ S₁⁻¹ = (R ∘ S₀ ∘ R) ∘ (R ∘ S₁ ∘ R) = R ∘ S₀ ∘ S₁ ∘ R`.
  have hF : R * S₀ * S₁ * R = (S₁ * S₀)⁻¹ := by
    calc R * S₀ * S₁ * R = R * S₀ * (R * R) * S₁ * R := by rw [hR, mul_one]
      _ = (R * S₀ * R) * (R * S₁ * R) := by simp only [mul_assoc]
      _ = (S₁ * S₀)⁻¹ := by rw [h₀, h₁, mul_inv_rev]
  refine ⟨?_, ?_, ?_, ?_⟩
  · -- `I₀ ∘ I₀ = (R ∘ S₀ ∘ R) ∘ S₀ = S₀⁻¹ ∘ S₀`
    rw [←mul_assoc, h₀, inv_mul_cancel]
  · -- `I₁ ∘ I₁ = S₁ ∘ (R ∘ S₁ ∘ R) = S₁ ∘ S₁⁻¹`
    calc S₁ * R * (S₁ * R) = S₁ * (R * S₁ * R) := by simp only [mul_assoc]
      _ = 1 := by rw [h₁, mul_inv_cancel]
  · -- `I₀ ∘ F ∘ I₀ = R ∘ S₀ ∘ S₁ ∘ (S₀ ∘ R ∘ S₀) = R ∘ S₀ ∘ S₁ ∘ R`
    calc R * S₀ * (S₁ * S₀) * (R * S₀) = R * S₀ * S₁ * (S₀ * R * S₀) := by
          simp only [mul_assoc]
      _ = (S₁ * S₀)⁻¹ := by rw [k₀, hF]
  · -- `I₁ ∘ F ∘ I₁ = (S₁ ∘ R ∘ S₁) ∘ S₀ ∘ S₁ ∘ R = R ∘ S₀ ∘ S₁ ∘ R`
    calc S₁ * R * (S₁ * S₀) * (S₁ * R) = (S₁ * R * S₁) * S₀ * S₁ * R := by
          simp only [mul_assoc]
      _ = (S₁ * S₀)⁻¹ := by rw [k₁, hF]
```

`intro` names the three hypotheses `hR`, $\mathtt{h}_0$ and $\mathtt{h}_1$. The proof first derives $S_0 \circ R \circ S_0 = R$ from $\mathtt{h}_0$, and the same for S_1 ($\mathtt{k}_0$ and $\mathtt{k}_1$). It uses two theorems of Mathlib about groups: an element whose product with x is 1 is the inverse of x, and an involution is its own inverse. `hF` then computes $F^{-1} = R \circ S_0 \circ S_1 \circ R$. Each of the four claims is a `calc`, a chain of equations, each with its reason: `rw [`$\mathtt{h}_0$`]`

rewrites with an equation, and `simp only [mul_assoc]` regroups a product, which the book does without saying so. The proof file ends with an example in which the hypotheses hold and R is not the identity: the two shears of Section 5.18 on $\mathbb{Z}/8 \times \mathbb{Z}/8$, with the flip $R(p, q) = (-p, q)$.

Fidelity note. The two steps and the involution are permutations of an arbitrary type of states.

Noether's theorem, and the square

An affine law (definition 5.5) acts on the tables of a set of cells `C`, with values in `ZMod n`:

```
@[book "def:affine-law"]
structure AffineLaw (C : Type*) (n : ℕ) where
  M : (C→ZMod n) ≃+ (C→ZMod n)
  b : C→ZMod n
namespace AffineLaw
variable {C : Type*} {n : ℕ}
def tick (L : AffineLaw C n) (p : C→ZMod n) : C→ZMod n := L.M p + L.b
def toDetLaw (L : AffineLaw C n) : DetLaw (C→ZMod n) := ⟨L.tick⟩
```

`M` is an additive bijection, `≃+` in Mathlib, and `b` is the source. `toDetLaw` reads the affine law as a deterministic law (definition 5.1), whose symmetries and conserved readings are those of definition 5.3:

```
@[book "def:tick"]
structure DetLaw (S : Type*) where
  tick : S→S
namespace DetLaw
variable {S : Type*} (L : DetLaw S)
def run (x : S) (n : ℕ) : S := L.tick^[n] x
@[book "def:law-symmetry"]
def IsSymmetry (s : S→S) : Prop := ∀ x, L.tick (s x) = s (L.tick x)
@[book "def:law-symmetry"]
def Conserved {V : Type*} (q : S→V) : Prop := ∀ x, q (L.tick x) = q x
```

Tallies, pairings and keeping a pairing are written with additive maps:

```
@[book "def:affine-law"]
abbrev Tally (C : Type*) (n : ℕ) := (C→ZMod n) →+ ZMod n
@[book "def:affine-law"]
abbrev Pairing (C : Type*) (n : ℕ) :=
  (C→ZMod n) →+ (C→ZMod n) →+ ZMod n
@[book "def:affine-law"]
def Pairing.Perfect {C : Type*} {n : ℕ} (P : Pairing C n) : Prop :=
  ∀ T : Tally C n, ∃! w, P w = T
@[book "def:affine-law"]
def AffineLaw.Keeps {C : Type*} {n : ℕ} (L : AffineLaw C n)
    (P : Pairing C n) : Prop :=
  ∀ p q, P (L.M p) (L.M q) = P p q
```

A pairing sends a table to a tally, so that `P w` is the tally $T_w = \langle w, \cdot \rangle$ and `P w p` is $\langle w, p \rangle$.

Theorem 5.6 Noether's theorem for affine laws

Let an affine law $F(p) = M(p) + b$ keep a pairing $\langle\,,\rangle$.

1. For every table w with $M(w) = w$, adding w to every state is a symmetry of the law, and the tally $T_w(p) = \langle w, p \rangle$ changes at every tick by exactly the source's share:

   $$T_w(F(p)) = T_w(p) + \langle w, b \rangle.$$

   In particular T_w is conserved when $\langle w, b \rangle = 0$.

2. If the pairing is perfect, every conserved tally is T_w for exactly one table w, and that w has $M(w) = w$ and $\langle w, b \rangle = 0$.

```
@[book "thm:noether"]
def noether : Prop :=
  ∀ (C : Type) [Fintype C] (n : ℕ) (L : AffineLaw C n) (P : Pairing C n),
    L.Keeps P →
    (∀ w, L.M w = w →
      L.toDetLaw.IsSymmetry (· + w) ∧
      (∀ p, P w (L.tick p) = P w p + P w L.b) ∧
      (P w L.b = 0 → L.toDetLaw.Conserved (P w))) ∧
    (P.Perfect → ∀ T : Tally C n, L.toDetLaw.Conserved T →
      (∃! w, P w = T) ∧ ∀ w, P w = T → L.M w = w ∧ P w L.b = 0)
```

Book	Lean
$F(p)$	`L.tick p`
F keeps the pairing	`L.Keeps P`
adding w to every state is a symmetry	`L.toDetLaw.IsSymmetry (· + w)`
$T_w(F(p)) = T_w(p) + \langle w, b\rangle$	`P w (L.tick p) = P w p + P w L.b`
T_w is conserved	`L.toDetLaw.Conserved (P w)`
T_w for exactly one table w	`∃! w, P w = T`

`(· + w)` is the function $p \mapsto p + w$. The proof follows the book's. For part 1, $\langle w, M(p)\rangle = \langle M(w), M(p)\rangle = \langle w, p\rangle$. For part 2, a conserved tally has $T(b) = T(F(0)) = T(0) = 0$, so M keeps it; the pairing is perfect, so $T = T_w$ for one w, and M fixes that w.

The law of the square moves each value one corner on, modulo 3:

```
def squareShift : (Fin 4 → ZMod 3) ≃+ (Fin 4 → ZMod 3) where
  toFun p i := p (i - 1)
  invFun p i := p (i + 1)
  left_inv p := by ext i; simp
  right_inv p := by ext i; simp
```

```
  map_add' p q := rfl
def squareLaw : AffineLaw (Fin 4) 3 := ⟨squareShift, 0⟩
def squareTotal : Tally (Fin 4) 3 :=
  AddMonoidHom.mk' (fun p => p 0 + p 1 + p 2 + p 3)
    (fun p q => by simp only [Pi.add_apply]; ring)
```

`squareShift` is the additive bijection $p \mapsto (i \mapsto p(i-1))$, with its inverse and the proofs that the two undo each other. `squareLaw` has source 0, and `squareTotal` is the total of the four values, with the proof that it is additive.

Corollary 5.7 Noether's theorem on the square

For the law of the square and the pairing $\langle p, q\rangle = \sum_i p(v_i)q(v_i)$, the tables fixed by the tick are the three constant ones; adding the same number at every corner is a symmetry; the tally paired with adding 1 at every corner is the total $T(p) = p(v_0)+p(v_1)+p(v_2)+p(v_3)$ modulo 3, and it is conserved; and every conserved tally is 0, T or $2T$.

```
@[book "cor:noether-square"]
def noetherSquare : Prop :=
  {w | squareLaw.tick w = w} =
    Set.range (fun c : ZMod 3 => fun _ : Fin 4 => c) ∧
  (∀ c : ZMod 3, squareLaw.toDetLaw.IsSymmetry (· + fun _ => c)) ∧
  dotPairing (Fin 4) 3 (fun _ => 1) = squareTotal ∧
  squareLaw.toDetLaw.Conserved squareTotal ∧
  ∀ T : Tally (Fin 4) 3, squareLaw.toDetLaw.Conserved T →
    T = 0 ∨ T = squareTotal ∨ T = 2 • squareTotal
```

The first line says that the fixed tables are exactly the constant ones, and `2 • squareTotal` is $2T$. The proof applies Theorem 5.6 to the square: the tick keeps the pairing $\sum_i p(v_i)q(v_i)$, that pairing is perfect, and a fixed table is constant. Its last step uses that a number

modulo 3 is 0, 1 or 2, which `decide` checks, as in the listing on `ZMod` above.

Fidelity notes. Theorem 5.6 is stated for every n. The case $n = 0$, in which `ZMod 0` is the integers, is included, so the Lean statement is slightly more general than the book's. In Corollary 5.7 the corners are `Fin 4`, numbered anticlockwise, and "the three constant ones" is stated as: the fixed tables are exactly the constant tables.

Discrete Stokes on a canvas

Chapters 5 and 6 use canvases of dimension two with a stored orientation. The library writes definition 3.1 for that case:

```
@[book "def:canvas"]
structure Canvas where
  V : Type
  E : Type
  F : Type
  [fintypeV : Fintype V]
  [fintypeE : Fintype E]
  [fintypeF : Fintype F]
  [decEqV : DecidableEq V]
  [decEqE : DecidableEq E]
  [decEqF : DecidableEq F]
  src : E→V
  tgt : E→V
  boundary : F→List (E × Bool)
  closed : ∀ f, IsClosedWalk src tgt (boundary f)
```

A canvas has finite types of vertices, edges and faces, whose values can be compared. The six fields in square brackets say so, and Lean finds them by itself. Each edge has a source and a target. Each face has its boundary cycle, with the proof that it is closed. The cycle is a list of edges, each marked `true` when the cycle runs along the edge's

stored direction and `false` when it runs against it. How a face reads an edge, and when the faces are read anticlockwise, are defined from the cycles:

```
def inc (e : K.E) (f : K.F) : ℤ :=
  ((K.boundary f).map fun st =>
    if st.1 = e then (if st.2 then 1 else -1) else 0).sum
structure Anticlockwise : Prop where
  nodup : ∀ f, ((K.boundary f).map Prod.fst).Nodup
  opposite : ∀ e f g b c, f ≠ g →
    (e, b) ∈ K.boundary f → (e, c) ∈ K.boundary g → b ≠ c
```

`inc e f` counts the times the boundary of `f` runs along `e`, less the times it runs against it: an entry of the book's incidence matrix. A canvas is anticlockwise when no face runs along an edge twice, and two different faces that share an edge read it in opposite directions, as faces drawn in the plane and read anticlockwise do. The operators of Section 5.17 follow, for a canvas `K` and values in a commutative ring `R`:

```
def diff (φ : K.V → R) : K.E → R := fun e => φ (K.tgt e) - φ (K.src e)
def curl (A : K.E → R) : K.F → R := fun f => Σ e, (K.inc e f : R) * A e
def div (E : K.E → R) : K.V → R := fun v =>
  Σ e, ((if K.src e = v then E e else 0) - (if K.tgt e = v then E e else 0))
def curlStar (B : K.F → R) : K.E → R := fun e => Σ f, (K.inc e f : R) * B f
```

Proposition 5.38 Discrete Stokes and divergence theorems

Let the canvas carry its stored orientation, each face read anticlockwise.

1. $\operatorname{curl} D = 0$ and $\operatorname{div}\operatorname{curl}^{*} = 0$: a difference has no curl, and an adjoint curl has no divergence.

2. *Stokes.* For a set S of faces, $\sum_{f \in S} \operatorname{curl} A(f)$ is the sum of A round the boundary of S: over the edges that lie on exactly one face of S, each read as that face reads it.

3. *Divergence.* For a set U of vertices, $\sum_{v \in U} \operatorname{div} E(v)$ is the sum of E over the edges with exactly one end in U, each read in the direction leaving U.

4. *Summation by parts.* Summing over the stored edges,

$$\sum_e D\phi(e)\, E(e) = -\sum_v \phi(v) \operatorname{div} E(v),$$
$$\sum_f \operatorname{curl} A(f)\, B(f) = \sum_e A(e) \operatorname{curl}^* B(e).$$

```
@[book "prop:discrete-stokes"]
def discreteStokes : Prop :=
  ∀ (K : Canvas) (R : Type) [CommRing R],
    (∀ φ : K.V→R, K.curl (K.diff φ) = 0) ∧
    (∀ B : K.F→R, K.div (K.curlStar B) = 0) ∧
    (K.Anticlockwise→∀ (S : Finset K.F) (A : K.E→R),
      Σ f ∈ S, K.curl A f =
        Σ e ∈ Finset.univ.filter (fun e =>
            (S.filter fun f => e ∈ (K.boundary f).map Prod.fst).card = 1),
          (Σ f ∈ S, (K.inc e f : R)) * A e) ∧
    (∀ (U : Finset K.V) (E : K.E→R),
      Σ v ∈ U, K.div E v =
        Σ e ∈ Finset.univ.filter (fun e => K.src e ∈ U ∧ K.tgt e ∉ U), E e -
          Σ e ∈ Finset.univ.filter (fun e => K.tgt e ∈ U ∧ K.src e ∉ U),
            E e) ∧
    (∀ (φ : K.V→R) (E : K.E→R),
      Σ e, K.diff φ e * E e = -Σ v, φ v * K.div E v) ∧
    (∀ (A : K.E→R) (B : K.F→R),
      Σ f, K.curl A f * B f = Σ e, A e * K.curlStar B e)
```

Book	Lean
$\operatorname{curl} D = 0$, $\operatorname{div}\operatorname{curl}^* = 0$	`K.curl (K.diff φ) = 0`, `K.div (K.curlStar B) = 0`
each face read anticlockwise	`K.Anticlockwise →`
the edges on exactly one face of S	`(S.filter ...).card = 1`
each read as that face reads it	`Σ f ∈ S, K.inc e f`
the edges with exactly one end in U, read leaving U	source in `U` and target outside, less the reverse

The proof of each part is a lemma of `Ch05/Stokes.lean`, and the theorem collects them:

```
@[book "prop:discrete-stokes"]
theorem discreteStokes : Statement.discreteStokes := fun K _ _ =>
  ⟨K.curl_diff, K.div_curlStar, K.stokes, K.sum_div, K.sum_diff_mul,
    K.sum_curl_mul⟩
```

$\operatorname{curl} D = 0$ telescopes: round a closed cycle, the differences $\varphi(\text{end}) - \varphi(\text{start})$ of its steps add up to zero. Summation by parts exchanges two finite sums, and $\operatorname{div}\operatorname{curl}^* = 0$ follows from it. Stokes exchanges the sums too, and then uses that the canvas is anticlockwise: an edge on two faces of S is read by them in opposite directions, so its two terms cancel. The two squares of Section 5.5 are an anticlockwise canvas, which `decide` checks:

```
example : Canvas.Anticlockwise
    { V := Fin 6, E := Fin 7, F := Fin 2
      src := ![0, 1, 2, 3, 1, 4, 5]
      tgt := ![1, 2, 3, 0, 4, 5, 2]
      boundary := ![[(0, true), (1, true), (2, true), (3, true)],
        [(4, true), (5, true), (6, true), (1, false)]]
      closed := by unfold IsClosedWalk; decide } :=
  ⟨by decide, by decide⟩
```

`![0, 1, 2, 3, 1, 4, 5]` is the function on `Fin 7` with those values. The vertices $v_0, \ldots, v_5$ are `0` to `5`, the edges $e_1, \ldots, e_7$ are `0` to `6`, and the right face runs against the shared edge e_2.

Fidelity note. The book asks, for the whole proposition, that the faces be read anticlockwise. Only Stokes uses it, and the Lean statement assumes it there alone, which makes the other three parts stronger. For an edge on exactly one face of S, the sum $\sum_{f \in S} \mathrm{inc}(e, f)$ has one term that is not zero, and it is how that face reads the edge.

How the repository checks itself

`#print axioms` lists the axioms a theorem depends on, through every theorem it uses:

```
#print axioms noether
```

Lean replies:

```
'Artform.noether' depends on axioms: [propext, Classical.choice, Quot.sound]
```

These three are the axioms of Mathlib's mathematics. `propext` says that two propositions that imply each other are equal. `Quot.sound` says that a quotient identifies the values that its relation relates; the principle `funext` of Part II follows from it. `Classical.choice` picks a value of any type that has one, which gives proof by contradiction. Every theorem of the library uses at most these three, and a check run on every change lists the axioms of every theorem and fails on any other.

Lean has a faster way to decide a statement, `native_decide`, which runs compiled code instead of the kernel's own computation. Its result is trusted only as far as the compiler is, and Lean

records that trust as an extra axiom of the theorem, so the check rejects it. The large computations of the library use `decide +kernel` instead, which the kernel itself checks, on data that a script computes beforehand. The sixty rotations of the icosahedron (proposition 12.8) are checked so.

An approval records two hashes. The first is of the text the author read: the statement, and the definitions it uses, as Lean prints them. The second is of their exact terms, as Lean stores them. If either changes, because a definition was edited or Lean prints it differently, the statement is no longer proved and goes back to the author.

GitHub runs the same commands on every change:

- `lake build -wfail` compiles the library and the files of this guide, and fails on any warning; the axioms of every theorem are then checked;
- `scripts/Report.lean` lists every declaration tied to the book, and `scripts/status.py -check` checks that each proof proves exactly its statement and that the status table is current;
- `scripts/check_tree.py` checks that no file contains `sorry` or another forbidden word;
- the guide's own check fails if this PDF is not built from the current sources.

Each release also runs `leanchecker`, a second checker, separate from the one that compiled the library, which reads every declaration again from the compiled files.

Following a proof, and reporting a problem

To follow a proof, open its file in VS Code and put the cursor after a tactic. The infoview shows the goal at that point, with its hypotheses. Holding the control key (on a Mac, the command key) and clicking a name opens its definition, in the library or in Mathlib, and hovering over a name shows its type and documentation.

A Lean statement may seem to say less or more than the book, or the book may seem wrong. The file `textbook/FINDINGS.md` records what has been found so far, and an issue on the repository's GitHub page reports something new. A statement is never weakened to make a proof go through. One that cannot be proved as the book prints it stays unproved, with the reason recorded, until the book is corrected.

Appendix. The book's statements in the repository

Every numbered statement of the book, with the Lean name of its proof, or of its statement while it has no proof, and its status when this guide was built. The names are in the namespace `Artform`, which is left out. The file `textbook/STATUS.md` of the repository is always current.

No.	Statement	Lean	Status
1.3	Uniqueness of rod alignment	`rodAlignmentUnique`	stated
1.10	The two tetractys interpretations coexist	`tetractysIntwits`	stated
1.16	The Artform category	`artformCategory`	stated
1.19	The semantic quotient	`semanticQuotient`	stated
1.27	Adjoints of reindexing	`adjoints`	stated
1.29	Sets form a topos	`finsetTopos`	stated
1.30	The Artform category is not a topos	`artformNotTopos`	stated
1.31	The laws of a rod	`rodLaws`	stated
3.4	Cube boundary assembled from squares	`cubeAssembly`	stated

No.	Statement	Lean	Status
3.11	Shared positive pattern	`sharedPositivePattern`	stated
3.12	Partial profile isomorphism	`partialProfileIso`	stated
3.22	Positive occurrences are atom images	`atomImageOccurrence`	stated
3.25	Averaged invariant metric	`averagedInvariantMetric`	stated
3.26	Perpendicularity from symmetry	`perpendicularityFrom Symmetry`	stated
3.27	Commuting compatible changes	`commutingProposals`	stated
3.29	Stabilization	`growthStabilization`	stated
3.32	Fair closure	`fairClosure`	stated
3.33	Symmetry of growth	`growthSymmetry`	stated
3.37	Finite Fubini–Tonelli	`finiteFubiniTonelli`	stated
3.39	Monotone convergence	`monotoneConvergence`	stated
3.40		`invariantMeasureOrbits`	stated
3.43	Receipts decide, and margins protect	—	planned
3.45	Reference marking along a comparison	`referenceMarkingPullback`	stated
4.2	Counting retained alternatives	`countLaw`	stated
4.4	Exit-level closure	`exitLevelClosure`	stated
4.5	Extension of compatible finite histories	`historyExtension`	stated

No.	Statement	Lean	Status
5.4	What a conserved reading can see	`orbitCriterion`	stated
5.6	Noether's theorem for affine laws	`noether`	stated
5.7	Noether's theorem on the square	`noetherSquare`	stated
5.8	The current of a tally	`tallyCurrent`	stated
5.11	The law decides what is kept	`lawDecides`	stated
5.13	Finite well-posedness	`finiteWellPosedness`	stated
5.16	Pose orbit	`poseTorsor`	stated
5.20	Connection and coordinates	`connectionCoordinates`	stated
5.23	Holonomy descent	`holonomyDescent`	stated
5.27	Pure gauge equals loop-flat	`pureGauge`	stated
5.28	Spanning-tree gauge fixing	`treeGaugeFixing`	stated
5.29	Classification of face-flat connections	`flatClassification`	stated
5.36	Gauge covariance passes to runs	`gaugeRuns`	stated
5.37	Five ticks read the outside	`fiveTicks`	stated
5.38	Discrete Stokes and divergence theorems	`discreteStokes`	stated
5.39	Reversing a tick of two steps	`twoStepReversal`	stated

No.	Statement	Lean	Status
5.40	Closed-window conservation	`closedWindow`	stated
5.42	What a strung run fixes	`runFixes`	stated
5.44	Hall; supplies and demands	`hall`	stated
5.45	A kept count is woven	`woven`	stated
5.46	Symmetries of a slot tapestry	`slotSymmetry`	stated
5.47	Symmetry of what may happen	`curie`	stated
5.48	Breaking needs a choice	`breaking`	stated
6.1	Reversibility	`reversible`	stated
6.2	Time reversal of the Maxwell tick	`maxwellReversal`	stated
6.3	Gauge covariance of the Maxwell tick	`maxwellCovariant`	stated
6.4	Gauss preservation	`gaussPreserved`	stated
6.5	The Maxwell shears keep the canonical pairing	`maxwellPairing`	stated
6.6	Gauss's law from the tally of gauge symmetry	`gaussNoether`	stated
6.7	A gauge change that varies from tick to tick	`gaugeDefect`	stated
6.9	Temporal links	`temporalLinks`	stated

No.	Statement	Lean	Status
6.10	Gauge covariance of a hop	`hopCovariance`	stated
6.11	What the two steps force	`maxwellShape`	stated
7.3	A kept count is woven: coherent laws	`qrWoven`	stated
7.4	The selected native probability law	—	planned
10.1	The colourless three-quark state	`mpEpsilon`	stated
10.2	Triality	`mpTriality`	stated
11.2	The two write families	`mtWrites`	stated
11.4	Blocks fill the table space	`mtBlocks`	stated
11.5	Averaging counts	`mtAverage`	stated
11.6	One-edge operators	`mtConvolution`	stated
11.8	Six native sign reads on the cube group	`mtNativeReads`	stated
11.11	The order is a presentation	`mtOrderWitness`	stated
11.12	Second quantization is a homomorphism	`mtGamma`	stated
11.14	Colourless states are the invariants of one vertex	`mtGaugeFix`	stated
11.15	Census of colourless states	`mtCensus`	stated
11.16	The hop is covariant	`mtHopCovariant`	stated

No.	Statement	Lean	Status
11.17	The coin's coefficients are minors	`mtMinors`	stated
11.18	The tick preserves the colourless sector	`mtTickGauss`	stated
11.19	Link moments	`mtMoments`	stated
11.20	The there-and-back tally	`mtThereAndBack`	stated
12.1	The eight invariants of a vertex with two slots	—	planned
12.2	Every extra state is located in one average	—	planned
12.3	The pair is bound more tightly than the quartet	—	planned
12.4	The spectrum of the tally on one edge	—	planned
12.5	The two-baryon states are ordered by spin	—	planned
12.6	The pair hop is the cause	—	planned
12.7	A symmetry that only a real triplet has	—	planned
12.8	The sixty rotations	`sixtyRotations`	stated
12.9	The tensor square of the icosahedral triplet	`a5Tensor`	stated
12.10	Models I and II share their zero-flux sector	—	planned
12.12	Triality, and the pair moment	—	planned
12.13	Orders from traces	`s1080Orders`	stated

No.	Statement	Lean	Status
12.14	The tensor squares of the Valentiner triplet	`s1080Tensor`	stated
12.15	Agreement with $SU(3)$ at two slots	—	planned
12.16	Every loop closes	`mmLoop`	stated
13.1	Colourless states of a path	—	planned
13.2	The string	—	planned
13.3	The screening number	—	planned
13.4	Where the string and screened levels cross	—	planned
13.5	Colourless states of the empty square	—	planned
13.6	The magnetic term in the electric basis	—	planned
13.7	The square to third order	—	planned
13.8	The third-order transfer	—	planned
13.9	What the transfer does	—	planned
13.10	A baryon on a ring and on the cube	—	planned
13.11	Census of a canvas	—	planned

Bibliography

[1] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. Theorem proving in Lean 4. https://lean-lang.org/theorem_proving_in_lean4/. With contributions from the Lean Community; consulted 2026-10-08.

[2] Jeremy Avigad and Patrick Massot. Mathematics in Lean. https://leanprover-community.github.io/mathematics_in_lean/. Consulted 2026-10-09.

[3] David Thrane Christiansen. Functional programming in Lean. https://lean-lang.org/functional_programming_in_lean/. Consulted 2026-10-09.

[4] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2–3):95–120, 1988.

[5] Thierry Coquand and Christine Paulin. Inductively defined types. In *COLOG-88*, Lecture Notes in Computer Science, pages 50–66. Springer, 1990.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, pages 625–635. Springer, 2021.

[7] Peter Dybjer. Inductive families. *Formal Aspects of Computing*, 6(4):440–465, 1994.

[8] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, 2020.